

Assignment 8: Embedding-based Search

Like in the previous assignment, you are a data engineer building a simple search engine for biomedical research abstracts. Your task is still to process PubMed abstracts, but you will be using a different approach: you will generate embeddings for the abstracts to allow for semantic search on them. You will also build your own retrieval-augmented generation (RAG) system to answer more complex queries, such as "What are the main precursors of lung cancer?"

The learning outcomes are:

- To understand how embeddings work and their uses.
- To learn how to work with vector databases.
- To learn about RAG and implement your own RAG system.

You will be given the following dataset (same as the previous assignment):

- An XML file (pubmed25n0001.xml) with a list of medical publications, with information such as author, year, title, and paper abstract.

1. Setup

Feel free to use the same packages as given in the previous assignment. The only additional package you will need is chromadb, a vector database. You can install it using pip:

```
pip install chromadb
```

You will be building off of your work in the previous assignment. Feel free to reuse your infrastructure.

2. Task

You will be inserting the abstracts into a vector database, then building a Flask app with an API for searching the abstracts.

Notice that we are trying to insert text into a vector database. Therefore, we will need some method of representing the text as a vector: an embedding. Generally, these vectors have hundreds to thousands of dimensions, and each word or phrase is mapped to a single vector of that dimension. Now, how to map each word to a vector is a difficult challenge, but we are interested in embedding functions that transform similar words and phrases into vectors close to each other. Therefore words such as "king" and "queen" are close together, while words such as "biology" might be farther away.

We will use this property of embeddings to help us search through our list of abstracts. We will first convert all of our abstracts into vectors using an embedding function and insert it into a database, and then convert any queries to that database to a vector using the same embedding function. We can then find the closest vectors to that query (and therefore, the most similar/relevant abstracts to that query). Chromadb is a vector database that directly supports such a task.

There are three parts to this assignment.

Part 1: Adding the embedded abstracts to the database

To start, as in the previous assignment, extract the xml file's PMID, ArticleTitle, and AbstractText; filter the abstracts using the same parameters as in a7. Next, create a chromadb instance; you may use the default embedding (all-MiniLM-L6-v2), as it is free (other proprietary embeddings such as OpenAI's have fees). Then, add each abstract to chromadb.

Adding the abstracts to ChromaDB may take a while because each abstract must be converted into an embedding and then stored in the vector database. In practice, adding a very large number of abstracts in a single call can be slow, memory-intensive, and harder to recover from if the process is interrupted.

One practical way to make this process more reliable is to add the abstracts in smaller batches, such as 500–1000 abstracts at a time. For example, instead of inserting all abstracts at once, the loading script could insert abstracts 0–999, then 1000–1999, and continue until all abstracts have been added.

Once the abstracts are added, you should then be able to query the database.

Part 2: Building the Flask app

Create a flask app that exposes a single endpoint: when provided a plaintext search query (a phrase, word, or sentence), it returns the top 5 closest abstracts. Your flask app can run on your local machine.

Part 3: Creating a RAG system based on PubMed articles database

Retrieval-Augmented Generation (RAG) is a technique that combines the strengths of a retrieval system with a large language model (LLM). Instead of relying solely on the LLM's pre-trained knowledge to answer a question, a RAG system first retrieves a set of relevant documents from an external database (in our case, the chromadb vector database you built in Part 1), and then passes those documents as context to the LLM alongside the original query. This allows the LLM to generate responses that are grounded in specific, up-to-date, or domain-specific information it may not have seen during training.

For this part of the assignment, we will use [OpenRouter](#), a unified API and platform that provides access to over 400+ LLM (including GPT, Claude, Llama, and Mistral) through a single account and API key. While most of LLM providers will charge money by a pay-as-you-go pricing model, OpenRouter have an advantage to access to [free models](#) with a really good performance for testing. As shown in the free-models documentation, you can use the OpenAI Python SDK to obtain responses from the LLM. See documentation on [OpenAI chat-completions](#) to improve your RAG.

Follow these steps to design and create your own RAG:

- Create an account at OpenRouter and create an API Key. Be careful with exposing API Keys, if working on GitHub this should be on a .env file and gitignored.
- Create a rag_response function that will take one open-ended query (e.g. "What medications are used to treat high blood pressure, and how do they help?") that will be used for (i) query the database you build in Part 1, (ii) use the query results as context that will be provided to the LLM, and (iii) finally return an augmented response to your query question.

Afterwards, you should answer the following questions in your Jupyter notebook:

1. Explain with details how did you build your RAG. How does the LLM access to the database? How did you ensure that it actually used the information from the database and no pre-trained information?
2. How do you think you can improve your RAG system in the future? Mention at least 2 ways of improvement. *Note: You don't need to implement them for the assignment.*

Note on API budget constraint: You are limited to 20 requests per minute (RPM), and 200–500 requests per day for new or free-tier users. This should not be a limitation to completing the assignment successfully, you just need to keep in mind that there is a budget.

Fallback option when OpenRouter doesn't work: Ollama

As a backup option, if you run into persistent issues with OpenRouter, such as service instability or model unavailability, you may use [Ollama](#) to run a local LLM instead. Ollama allows you to download and run open-source models locally on macOS, Windows, and Linux. After installing Ollama, you can download a model from the [Ollama model library](#). You can then call the model from Python using the official [Ollama Python library](#), installed with `pip install ollama`. The RAG design remains the same: retrieve relevant abstracts from ChromaDB, provide them as context to the model, and generate an answer grounded in the retrieved abstracts.

3. Submission Guidelines

You will submit this assignment via Gradescope.

You must submit the following:

- 1) Implement your solution in a Jupyter notebook and submit that file. If you additionally used a Python script to load the data into ChromaDB, also submit that file.
- 2) Answer the two questions in part 3 using markdown inside your Jupyter notebook. Keep the answers together so that they are easy to find and select on Gradescope.
- 3) Implement your Flask app using `app.py` for the default embedding and submit that file.

Use of LLMs

The point of this assignment is to learn how to work with embeddings and RAG. Therefore you should not ask the LLM to directly solve these tasks for you, i.e. you should not paste any part of the assignment prompt into the LLM and have it generate a solution.

You may use an LLM to answer specific questions you have about the topics covered in this assignment (e.g., Flask, embeddings, chromadb, etc.). You may also use an LLM to help debug specific problems with your code. For example, you may ask the LLM for help if you are having trouble with getting any of the tools running.

If you use an LLM to help you with this assignment, you must include in a comment in the submitted notebook where you used them.